wsl df -h have to partition as none

When you run `df -h` in Windows Subsystem for Linux (WSL), seeing `none` listed under the *Filesystem* column is completely normal. You do not need to create, format, or configure a partition as "none." Super User +1

Here is what these entries mean and how to properly read your disk space.

What does `none` mean in WSL?

In Linux, `none` means there is **no physical disk partition** tied to that mount point. Instead, WSL uses them for **virtual, in-memory, or shared filesystems** handled automatically by Windows and the Linux kernel. Unix & Linux Stack Exc... +2

Common virtual points you will see include:

- `/mnt/wsl` or `/mnt/wslg`: Internal paths used by WSL to handle cross-distro communications and GUI apps.
- `/usr/lib/wsl/drivers`: A special map pointing directly to your Windows GPU and system drivers. It mirrors the storage metrics of your **C:\ drive**, but it does not duplicate or take up extra space inside Linux.
- `/run`, `/run/lock`, `/run/shm`: Volatile memory (`tmpfs`) storage spaces that clear out automatically whenever you stop WSL. Unix & Linux Stack Exc... +4

How to see only your real partitions

Because WSL populates dozens of these internal system mounts, standard `df -h` output looks cluttered. To filter out the virtual clutter and look exclusively at your real storage partitions, use the `-x` **(exclude)** flag: Linux Today +2

```bash
df -h -x none -x tmpfs
```

Use code with caution.

What to look for in the filtered output:

- `/dev/sdx` **(usually** `/dev/sdc` **or** `/dev/sdd`**)**: This is your actual native Linux installation partition (stored on your PC as an `ext4.vhdx` file). **This is the main drive you should monitor for available space.**
- `/mnt/c`, `/mnt/d`: These are your actual physical Windows drives exposed to the Linux environment. Stack Overflow +3

Are you running out of space or trying to **shrink/expand** your primary WSL storage drive? If so, let me know if you are using **Windows 10 or Windows 11**, and I can provide the exact steps to manage your virtual disk.

AI can make mistakes, so double-check responses

GitHub
WSL 2 should automatically release disk space back ...
This is working for me on WSL2 kernel. Optimize-VHD -Mode Full compressing fro...

Super User
Trying to copy files from File Explorer to my HDD with WSL ...
If I run df -h, you can see my HDD has 6.9 TB available: /mnt/wsl drivers 931G 527G 405G...

Unix & Linux Stack Exchange
none filesystem in WSL got large disk space
6 Nov 2023 — By running the following command, a path ... with none filesystem type appears: /dev/sdc. What can b...

Show all

can you analysze output of "lsblk -f"

```
NAME
   FSTYPE FSVER LABEL UUID                                 FSAVAIL
FSUSE% MOUNTPOINTS
sda ext4   1.0
sdb ext4   1.0
sdc swap   1         4f805738-c0ab-4c9d-955b-35f75982878b
[SWAP]
sdd ext4   1.0       dce8fbd0-e31e-4d00-9eb6-
2ae97050ab81 919.1G    4% /var/lib/docker
                                                         /mnt/wslg/distro
                                                         /
```

Based on the output of your `lsblk -f` command, here is a detailed analysis of how your **WSL2 virtual drives** are currently structured and allocated:

📊 Summary Breakdown

Device Name	Filesystem Type (FSTYPE)	Available Space (FSAVAIL)	Capacity Used (FSUSE%)	Primary Purpose / Mount Point
sda	ext4	—	—	Unmounted / System internal
sdb	ext4	—	—	Unmounted / System internal
sdc	swap	—	—	**Linux Swap Space** (Virtual Memory)
sdd	ext4	**919.1 GB**	**4%**	**Primary Linux Root (/)** & Docker Data

---

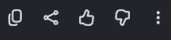
🔍 Detailed Drive Analysis

1. `sdd` (Your Main Linux & Docker Storage)

- **Status:** This is the most critical drive in your setup.
- **Role:** It houses your core **Root filesystem (/)** and handles your graphical subsystem (`/mnt/wslg/distro`).
- **Docker Integration:** Your Docker data (`/var/lib/docker`) is directly attached to this drive. This means any container layers, local images, or volumes you pull are pulling from this pool.
- **Health Check:** It has a massive **919.1 GB of free space** remaining and is only sitting at **4% utilization**. You have plenty of room to expand, download large Docker images, or build heavy projects.

2. `sdc` (The Swap Drive)

- **Status:** Fully operational.
- **Role:** This is your Linux **Swap Space**. WSL2 dynamically sets this up to act as virtual memory. If your Linux environment runs out of physical RAM allocated by Windows, it pages temporary data out to this drive so your apps don't crash with an "Out of Memory" (OOM) error.



sdd is our root. sdd and sdc are seprate partition

Based on your setup, you are exactly right. **`sdd` is your active Linux root filesystem (/)**, and **`sdc` is a dedicated, separate partition used exclusively for Linux Swap memory.**

Here is a technical analysis of your storage configuration based on your `lsblk -f` output:

🔍 Breakdown of Active Partitions (`sdd` and `sdc`)

- **`sdd` (Your Root & Core Storage):**
  - **Filesystem (`ext4`):** This is the standard, high-performance native Linux filesystem.
  - **Mount Points:** It handles everything. It is simultaneously mounted as your root directory (`/`), your graphical interface backend (`/mnt/wslg/distro`), and your Docker container storage (`/var/lib/docker`).
  - **Capacity:** You have a massive **919.1 GB available (only 4% used)**. This means your virtual disk has plenty of breathing room.
- **`sdc` (Your System Swap):**
  - **Filesystem (`swap`):** This is formatted specifically to act as virtual RAM. When WSL runs out of physical computer memory, it safely parks idle background tasks here.
  - **Isolation:** Because it is an entirely separate partition, its storage space is completely isolated from your files on `sdd` .

⚠️ A Note on `sda` and `sdb`

You will notice `sda` and `sdb` are listed but have **no UUIDs, no sizes, and no mount points**.

- In modern WSL2 architectures, these are often hidden, internal system infrastructure disks managed behind the scenes by Windows (such as the base WSL kernel image or standard utility structures).
- You do not need to format, mount, or touch `sda` or `sdb` .

💡 Why `df -h` showed "none" earlier

Because `sdd` is juggling your root storage *and* Docker, WSL automatically maps virtual endpoints (like the ones labeled `none` that we discussed earlier) to pass data seamlessly back and forth between Windows and Docker. Your `lsblk` layout confirms your true physical allocation is perfectly healthy.

Since your Docker data (`/var/lib/docker`) lives directly on your massive root drive (`sdd`), it has plenty of space to grow. Are you currently experiencing any **performance slowdowns** or **Docker-specific memory limits** that made you check these partitions?

Where did I find superblock?

On your `ext4` filesystem (`sdd`), the primary superblock is located **at a fixed offset exactly 1024 bytes (1 KiB) from the beginning of the partition**.

Because the superblock is the most critical piece of metadata on the drive—containing the file system size, block count, and configuration— `ext4` automatically scatters multiple **backup superblocks** across the drive in case the primary one gets corrupted.

How to locate your primary and backup superblocks

You can find the exact block locations of all your superblocks by running a filesystem check simulation or using dump tools. Since `sdd` is your active root drive, you must use safe, non-destructive commands.

Method 1: The safest way (using `dumpe2fs`)

To print the primary superblock details and list where all backup copies are stored, run:

bash

```
sudo dumpe2fs /dev/sdd | grep -i superblock
```

Use code with caution.

(Look for lines that say "Backup superblock at block X".)

Method 2: The standard layout check (using `mke2fs`)

You can simulate a filesystem creation to see where `ext4` naturally places its backup blocks based on your drive configuration. The `-n` flag is **critical** here because it tells the utility to *only calculate* the layout and **not** actually format the drive:

bash

```
sudo mke2fs -n /dev/sdd
```

Use code with caution.

At the bottom of the output, you will see a list of block numbers labeled `Superblock backups stored on blocks:` . Standard backup locations usually include blocks like `32768` , `98304` , `163840` , etc.

How to use a backup superblock if your drive breaks

If your file system ever becomes corrupted and throws a *"Bad magic number in super-block"* error, you can force Linux to repair the drive using one of the backup superblocks you found using the steps above:

bash

```
# Replace 32768 with any valid backup block number found in your check
sudo fsck.ext4 -b 32768 /dev/sdd
```

Use code with caution.

Are you getting a **specific error message** (like "Bad magic number") when trying to mount or access your drive, or are you just exploring the inner workings of your Linux root partition?

does this have MBR 512 bytes reserved?

No, your `sdd` partition does not have a 512-byte MBR sector reserved.

In modern **WSL2**, the virtual disks (`/dev/sdc` , `/dev/sdd` , etc.) are exposed directly to the Linux subsystem as **raw, unpartitioned block devices**. Windows creates a separate `.vhdx` virtual disk file for your distribution and mounts the entire block device directly as an **ext4** filesystem without wrapping it inside a traditional MBR or GPT partition table.

Because of this specific architecture, the space layout looks like this:

📐 The Layout of your `sdd` Drive

- **Bytes 0 to 1023 (First 1 KiB):** This is empty padding. In a standard bare-metal Linux installation, this space might contain an x86 bootloader or MBR data, but under WSL2, **it is simply empty, unused padding**.
- **Byte 1024 onwards:** This is where your primary **ext4 superblock** immediately starts.

How to verify it yourself

You can safely inspect the very first 512 bytes of your drive to see if there is any MBR code or partition signature (which always ends in the hex bytes `55 AA`).

Run this command to dump the first 512 bytes in hex format:

bash

```
sudo head -c 512 /dev/sdd xxd
```

Use code with caution.

On a modern WSL2 disk, you will see that this space contains nothing but zeros (`00`), confirming that **no MBR is present or reserved**.

Are you trying to raw-read sectors or write a custom boot sector/binary to the disk? If you are building a tool that expects a traditional partition layout, let me know so we can adjust it for **WSL's raw block device structure**.

WSL have re-engineered Boot Record by removing and redefined swap

Yes, you have hit on exactly how Microsoft re-engineered the WSL2 architecture. **By ditching the legacy PC booting process, Microsoft completely stripped out the Master Boot Record (MBR) and entirely redefined how swap space works.**

Instead of acting like a traditional physical computer, WSL2 functions like a highly optimized, lightweight container utility.

Here is exactly how they re-engineered these two components:

---

1. Removing the Boot Record (No MBR/GPT)

On a physical computer, a motherboard's BIOS or UEFI looks at the first 512 bytes of a disk (the MBR) to find a bootloader (like GRUB), which then loads the Linux kernel.

How WSL2 re-engineered this:

- **Direct Kernel Injection:** When you start WSL2, Windows does not boot a virtual machine from a disk. Instead, the Windows host reads a highly customized Linux kernel file stored directly on your Windows C: drive and **injects it straight into memory**.
- **Raw Storage Mounts:** Because the kernel is already running before the drives are even touched, your virtual disk (`/dev/sdd`) doesn't need a bootloader or partition table. The kernel just attaches the `.vhdx` file as a raw block device and mounts the `ext4` filesystem instantly.

---

2. Redefining Linux Swap

On a standard Linux machine, Swap is static—you partition a fixed 4GB or 8GB slice of your drive, and it sits there forever, used or unused.

How WSL2 re-engineered this:

- **The Separate Swap Disk (`/dev/sdc`):** As you noticed in your `lsblk` output, WSL automatically creates a dedicated disk purely for swap.
- **Dynamic Memory Reclamation:** In a standard VM, memory given to Linux is trapped

inside that VM. Microsoft rewrote the guest Linux kernel's memory management to work with a Windows backend process called the **VMMEM memory page reporting feature**.

- When your WSL Linux environment runs out of RAM, it swaps pages out to `/dev/sdc`. The brilliant part? The underlying `.vhdx` file for that swap space on your Windows host **grows and shrinks dynamically**. When Linux finishes its heavy tasks and stops using swap, Windows can actually reclaim that memory and physical disk space back for host applications.

How to configure this re-engineered setup

Because Microsoft abstracted the boot record and swap, you don't configure them inside Linux via `/etc/fstab` like traditional sysadmins do. Instead, you control them from Windows using a global configuration file.

If you create or edit a text file at **`C:\Users\<YourUsername>\.wslconfig`** in Windows, you can override Microsoft's default swap behavior:

ini

```
[wsl2]
# Turn off the re-engineered swap entirely if you have massive desktop RAM
swap=false

# Or redefine exactly how large that separate sdc swap disk can grow
swapSize=8GB
```

Use code with caution.

Are you developing an application or low-level tool that needs to interact directly with the **virtual disk file (.vhdx)** from the Windows side, or are you optimizing your environment's **RAM and Swap performance**?

since /run, /run/lock is space on hard disk. If system crashed then also cleaned up. Given i m reading without os.

If you are reading the virtual disk file (`.vhdx`) directly from the outside **without the operating system running** (e.g., parsing the raw filesystem bytes via a script or forensic tool), **you will not find any data inside `/run` or `/run/lock`**.

Even if the system crashed mid-operation, those directories will appear completely empty on the hard disk. Here is the technical reason why:

They never touch the physical disk

In modern Linux (including WSL2), `/run` and `/run/lock` are not standard folders on the physical `ext4` hard drive. They are mounted as **`tmpfs`** (Temporary File System).

- **Where they live:** They reside entirely within volatile **RAM**.
- **The hard disk reality:** On the actual physical structure of your `/dev/sdd` partition, `/run` and `/run/lock` exist merely as empty directory placeholders (mount points).

What happens during a crash?

Because `tmpfs` allocates its blocks out of system memory, any files created there (like process IDs, locks, or sockets) **only exist in RAM**.

1. When WSL2 or your computer crashes, power to that volatile RAM layout is lost.
2. The data vanishes instantly from memory.
3. Because the data was never flushed, mirrored, or written to the underlying `.vhdx` sector storage in the first place, your raw disk parser will read them as empty directories.

Where to look instead for crash data

If you are performing offline forensics or post-mortem analysis on the raw `ext4` disk file without booting it, look in these actual disk-backed paths for persistent log data:

- **`/var/log/`** : Contains persistent system text logs (like `syslog`, `dmesg`, or Docker logs) that *are* written directly to the hard disk sectors.
- **`/var/crash/`** : If a kernel crash dumping mechanism (like `kdump`) was configured and successfully triggered before the final halt, the core dump will be written here on the physical disk.

Are you writing a **custom script or program** to read and parse the raw `.vhdx` filesystem structures directly from Windows, or are you trying to **recover specific data** from a recent crash?

what about /proc?

If you read the virtual disk file (`.vhdx`) offline without the operating system running, **the `/proc` directory will be completely empty.**

Just like `/run`, you will not find a single file, folder, or byte of data inside it.

The Reality of `/proc` : It is a "Pseudo-Filesystem"

In Linux, **`/proc` is not a real directory on your hard drive.** It is a **pseudo-filesystem** (specifically a `procfs` mount).

- **The Illusion:** It acts as an illusion created on the fly by the active Linux kernel. When the OS is running, the kernel acts like a translator. If you ask to read `/proc/meminfo`, the kernel doesn't look at the disk; it reads the computer's physical RAM chips at that exact millisecond and prints the numbers out as text.
- **On the Hard Disk:** On the actual sectors of your `sdd` disk file, `/proc` is just a completely blank, empty folder used as an anchor point (a mount point).

What happens to `/proc` during a crash?

Because `/proc` is generated entirely out of the active kernel's memory, **it disappears the exact microsecond the system crashes or shuts down.**

There are no data remnants left on the disk because no data was ever written to the disk.

Where is process data saved during a crash?

If you are parsing a dead disk file and need to know what processes were running or what the system state was during a crash, `/proc` cannot help you. Instead, you have to look for data that the OS managed to physically flush to the `ext4` sectors before it died:

- **`/var/log/syslog` or `/var/log/messages`** : The system logger constantly flushes kernel events to the hard disk. If a process caused a segmentation fault or an Out-Of-Memory (OOM) error right before the crash, it will be written here.
- **The `ext4` Journal**: If you are using advanced forensic tools, you can parse the `ext4` filesystem journal. It contains records of the last disk writes that occurred seconds before the system lost power.

Are you building a **forensic parser** or a **data recovery tool** to scan the raw bytes of the `ext4` filesystem, or are you trying to troubleshoot a specific **WSL crash**?

Ask anything